

Object-Oriented Programming

Unit 1: Abstract Data Types (ADTs)

MAKAUT B.Tech IT | Exam-Ready Notes with Code Examples

1. What is an Abstract Data Type?

An **Abstract Data Type (ADT)** is a mathematical / logical model of a data structure. It defines **what** operations can be performed on a collection of data and **what behaviour** those operations guarantee — without specifying **how** the data is stored in memory or **how** the operations are implemented internally.

In other words, an ADT is defined purely by its **interface** (the set of operations and their semantics), not by its implementation. It is called “abstract” because the user of the ADT only needs to know *what* it does, never *how* it does it — the implementation is hidden inside a black box.

Key characteristics of an ADT:

- Defines a set of **values** (the possible states of the data) and a set of **operations** on those values.
- Does **not** specify how data is organized in memory (array, linked list, tree, etc.).
- Does **not** specify the algorithms used to implement the operations.
- Gives an **implementation-independent** view — the same ADT can be implemented in many different ways.
- Achieved in practice through **encapsulation** (bundling data + operations) and **information hiding**.

Data Structure vs. Abstract Data Type

A common exam distinction: a **data structure** is a concrete way of organizing data in memory (e.g. an array or a linked list). An **ADT** is the abstract specification of behaviour (e.g. “Stack”: push, pop, peek, isEmpty). The same ADT (Stack) can be implemented using different data structures (array-based stack or linked-list-based stack) — the user's code does not change either way, because it only depends on the ADT's interface.

Aspect	Data Structure	Abstract Data Type
Focus	How data is stored	What operations are available
View	Implementation-level	Interface-level (black box)
Example	Array, Linked List, Tree	Stack, Queue, List, Set, Map
Changeable?	Implementation can vary	Specification stays fixed

2. Specifying an ADT

To fully specify an ADT, we describe two things:

- **Abstract state space** — the set of abstract values the ADT can represent (e.g. for a Text ADT, the abstract state is “a sequence of characters”).

- **Operations** — each operation is specified using **pre-conditions** (what must be true before it runs) and **post-conditions** (what will be true after it runs), independent of any implementation.

This specification style (pre/post-conditions on an abstract state) lets us reason about correctness without ever looking at code. Example (Text ADT, specified abstractly):

```
ADT Text
  abstract state: a finite sequence of characters

  operation insert(pos, c)
    pre: 0 <= pos <= length(text)
    post: a character c has been inserted into the
          sequence at index pos; all characters at or
          after pos are shifted one position to the right

  operation delete(pos)
    pre: 0 <= pos < length(text)
    post: the character at index pos has been removed;
          all later characters shift one position left

  operation length()
    post: returns the number of characters currently
          in the sequence
```

3. How to Implement an ADT

Once an ADT is specified abstractly, we implement it by choosing a **concrete data structure** (e.g. an array, a string, a linked list) to actually hold the data, and writing code for every operation using that structure. Implementing an ADT always requires defining three things precisely — this is the most frequently asked theory question in this unit:

(a) Concrete state space

The set of **all possible values** the chosen concrete representation can take, regardless of whether they are “valid”. It is entirely **implementation-dependent** — it changes if you change the underlying data structure.

(b) Concrete invariant

A **property that must hold for every valid state** of the concrete representation. Not every value in the concrete state space is meaningful — the invariant carves out exactly the subset of states that represent legal ADT values. It must be true before and after every operation.

(c) Abstraction function

A **mapping AF: concrete state → abstract state**. It tells us which abstract value a given (invariant-satisfying) concrete state represents. The abstraction function is what lets us prove an implementation is correct: we check that every operation, performed on the concrete representation, produces a new concrete state whose abstraction matches what the abstract specification promised.

***Exam tip — remember the chain:** Abstract specification (pre/post-conditions) → choose concrete representation → define concrete state space → define concrete invariant (which states are legal) → define abstraction function (what abstract value each legal state represents) → implement each operation so it preserves the invariant and matches the abstract specification via the abstraction function.*

4. Worked Example: Stack ADT (Array-Based)

This is the standard textbook example used to illustrate concrete state space, concrete invariant and abstraction function together.

Concept	Definition for an array-based Stack
Concrete state space	All possible arrays of fixed capacity N that can hold elements of type T , together with an integer
Concrete invariant	There exists an integer 'top' such that $-1 \leq \text{top} < N$, and only indices $0..top$ hold meaningful (pus
Abstraction function	$\text{AF}(\text{array}, \text{top}) =$ the sequence $[\text{array}[\text{top}], \text{array}[\text{top}-1], \dots, \text{array}[0]]$ read from top to bottom, repr

Java implementation:

```
public class ArrayStack<T> {
    private Object[] data;    // concrete representation
    private int top;          // -1 means empty stack

    // Concrete invariant:  $-1 \leq \text{top} < \text{data.length}$ 
    // Abstraction function:
    //    $\text{AF}(\text{this}) =$  sequence  $[\text{data}[\text{top}], \text{data}[\text{top}-1], \dots, \text{data}[0]]$ 

    public ArrayStack(int capacity) {
        data = new Object[capacity];
        top = -1;
    }

    public boolean isEmpty() {
        return top == -1;
    }

    public boolean isFull() {
        return top == data.length - 1;
    }

    public void push(T value) {
        if (isFull()) throw new RuntimeException("Stack overflow");
        data[++top] = value;          // invariant preserved
    }

    @SuppressWarnings("unchecked")
    public T pop() {
        if (isEmpty()) throw new RuntimeException("Stack underflow");
        return (T) data[top--];      // invariant preserved
    }

    @SuppressWarnings("unchecked")
    public T peek() {
        if (isEmpty()) throw new RuntimeException("Stack is empty");
        return (T) data[top];
    }

    public int size() {
        return top + 1;
    }
}
```

Notice how **push** and **pop** only ever move *top* in a way that keeps $-1 \leq \text{top} < \text{data.length}$ true (the invariant), and the abstract sequence returned by the abstraction function changes exactly as the Stack ADT specification promises.

5. Worked Example: Text ADT (String-Based)

The Text ADT models an editable sequence of characters — the classic example used to illustrate “implementing operations” in this syllabus.

Concept	Definition for a String/char-array-based Text
Concrete state space	All possible character arrays (of some current length) that can store characters.
Concrete invariant	There is no restriction on length (it can grow); the array always holds exactly 'length' valid characters.
Abstraction function	$\text{AF}(\text{charArray}, \text{length}) = \text{the character sequence } \text{charArray}[0..\text{length}-1]$, representing the abstract

Java implementation:

```
public class Text {
    private char[] chars;    // concrete representation
    private int length;      // number of valid characters

    // Concrete invariant: 0 <= length <= chars.length
    // Abstraction function:
    //   AF(this) = the character sequence chars[0..length-1]

    public Text() {
        chars = new char[16];
        length = 0;
    }

    public int length() {
        return length;
    }

    public boolean isEmpty() {
        return length == 0;
    }

    // pre: 0 <= pos <= length()
    // post: c has been inserted at index pos; later chars shift right
    public void insert(int pos, char c) {
        if (pos < 0 || pos > length)
            throw new IndexOutOfBoundsException();
        ensureCapacity(length + 1);
        for (int i = length; i > pos; i--)
            chars[i] = chars[i - 1];
        chars[pos] = c;
        length++;
    }

    // pre: 0 <= pos < length()
    // post: character at pos removed; later chars shift left
    public void delete(int pos) {
        if (pos < 0 || pos >= length)
```

```
        throw new IndexOutOfBoundsException();
    for (int i = pos; i < length - 1; i++)
        chars[i] = chars[i + 1];
    length--;
}

private void ensureCapacity(int needed) {
    if (needed <= chars.length) return;
    char[] bigger = new char[chars.length * 2];
    System.arraycopy(chars, 0, bigger, 0, length);
    chars = bigger;
}

@Override
public String toString() {
    return new String(chars, 0, length);
}
}
```

6. List ADT

A List ADT stores elements in a linear (sequential) order and is typically specified by the following standard operations:

- **get(pos)** — return the element at a given position.
- **insert(pos, value)** — insert an element at any position.
- **remove(value)** — remove the first occurrence of a value.
- **removeAt(pos)** — remove the element at a specific position.
- **replace(pos, value)** — replace the element at a position with a new value.
- **size()** — return the number of elements currently stored.
- **isEmpty()** / **isFull()** — boundary checks.

The List ADT can be implemented using an **array** (fast random access, slow insert/delete in the middle) or a **linked list** (slow random access, fast insert/delete anywhere) — a good example of how the same ADT specification supports multiple, very different implementations.

7. General Operations on Containers

Regardless of which specific ADT is being discussed, most containers support a common baseline of operations:

- **Container-level:** create, destroy, copy, split, union (merge), intersection.
- **Object-level:** get size, check emptiness, insert, search/membership check, remove, clear (remove all).

An ADT may also define the relationship between stored objects, e.g. no relation (Set/Map), linear ordering (List, Stack, Queue), hierarchical ordering (Tree), or adjacency relation (Graph). This is a useful way to classify ADTs for exam recall.

8. Advantages and Disadvantages of ADTs

Advantages	Disadvantages
Encapsulation of data + operations into one unit	Adds overhead (extra abstraction layers)
Abstraction hides implementation complexity	Can be complex to design for large structures
Data-structure independence (swap implementation freely)	Learning curve for correct usage
Information hiding protects data integrity	May be less flexible for very specialised needs
Modularity: ADTs combine to form complex structures	Additional development cost/time

9. Likely Semester-Exam Questions

Q1. Define an Abstract Data Type. How is it different from a data structure?

An ADT is a model that defines a set of values and operations on those values without specifying implementation; a data structure is the concrete way the data is actually organized in memory. The same ADT can have several different data-structure implementations.

Q2. What are concrete state space, concrete invariant, and abstraction function? Explain with an example.

Concrete state space = all possible values the chosen representation can hold. Concrete invariant = the property that must hold for all **valid** states. Abstraction function = the mapping from a valid concrete state to the abstract value it represents. Example: array-based Stack — see Section 4 table.

Q3. Implement a Stack ADT using an array. State its invariant and abstraction function.

See the ArrayStack Java implementation in Section 4, with invariant " $-1 \leq \text{top} < \text{capacity}$ " and abstraction function mapping the array/top pair to the sequence `data[top]`, `data[top-1]`, ..., `data[0]`.

Q4. Why is an ADT called "abstract"?

Because it presents only an interface (what operations exist and what they do) and hides all implementation details (how they are done) — the user interacts with a black box.

Q5. List the standard operations of a List ADT.

`get`, `insert`, `remove`, `removeAt`, `replace`, `size`, `isEmpty`, `isFull` — see Section 6.

Q6. What role does the abstraction function play in proving correctness of an ADT implementation?

It formally connects the concrete representation to the abstract specification. To prove an operation correct, we show that starting from a concrete state satisfying the invariant, the operation produces a new concrete state (also satisfying the invariant) whose image under the abstraction function matches what the abstract specification's post-condition requires.

Q7. Give one advantage and one disadvantage of using ADTs.

Advantage: implementation independence — you can change the underlying data structure without changing client code. Disadvantage: the extra abstraction/encapsulation layers can add performance overhead.

End of Unit 1 notes. Sources synthesised: University of Waterloo (ADT theory & container operations), Code World / arnab7070 (concrete state space, invariant, abstraction function, Text ADT example), plus original Java code examples and exam-style Q&A; written for this syllabus.